Technical Documentation

N-ATLaS Health QA: architecture, setup, reproduction and usage.

1. Overview

N-ATLaS Health QA is a LoRA adapter that specialises N-ATLaS, Nigeria's open language model, for African medical multiple-choice questions. It was built for Problem Statement 3 (Sectoral Fine-Tuning) in the health sector. The project is one reproducible pipeline: prepare the data, score base N-ATLaS, fine-tune, score again on the same questions, and publish the adapter with its measured results.

- Model: https://huggingface.co/Emar7/N-ATLaS-Health-QA-LoRA-Powered-by-Awarri
- Code: https://github.com/Emart29/emart-health

Model	Correct	Accuracy	95% confidence interval (Wilson)
Base N-ATLaS (NCAIR1/N-ATLaS)	252 / 500	50.4%	46.0% to 54.8%
N-ATLaS + this adapter	303 / 500	60.6%	56.2% to 64.8%
Chance (random guessing)		21.2%	

The fine-tuned model was more accurate than base N-ATLaS by 10.2 percentage points on the same 500 questions. The exact McNemar p-value is 1.4 x 10^-6, so the difference is statistically significant at the 0.05 level.

2. Architecture

There are two parts. The base model N-ATLaS (8 billion parameters, Llama architecture) is loaded unchanged from Hugging Face in 4-bit precision. The adapter adds low-rank weight updates to the attention and MLP projection layers: 41,943,040 trainable parameters, which is 0.52% of the 8.07 billion parameters in the model. At inference the adapter is loaded on top of N-ATLaS with the PEFT library. Nothing else is in the loop: no retrieval, no second model, no external API.

Step	What happens
1. Config	Model, dataset, seed 42, LoRA and training settings. All fixed in the script.
2. Data	Load AfriMed-QA v2, keep single-answer multiple-choice questions, parse the options and the answer key, drop duplicates, build each prompt with the N-ATLaS chat template, hold out 500.
3. Base score	Load N-ATLaS in 4-bit and score it on the 500 questions.
4. Fine-tune	Attach a LoRA adapter and train it with QLoRA. The base weights are frozen.
5. Re-score	Score the fine-tuned model on the same 500 questions. Compute intervals and McNemar's test.
6. Demo	Generate one free-text answer as a sanity check.
7. Publish	Save the adapter, write the model card from the measured numbers, push to Hugging Face.

3. Repository contents

File	Purpose
natlas_pipeline.py	The whole pipeline as one script. This is the source of truth.
natlas_health_kaggle.ipynb	The same code as a notebook, generated from the script. Used for the reported run.
natlas_health_demo.ipynb	Live demo: loads N-ATLaS, then the published adapter, and answers the same questions.
modal_natlas_health.py	Optional launcher that runs the script on a Modal GPU.
natlas_health_out/benchmark_summary.json	Both accuracies, intervals, p-value, split method, settings.

File	Purpose
natlas_health_out/benchmark_500_predictions.csv	One row per benchmark question with both models' answers.
natlas_health_out/benchmark_by_specialty.csv	Accuracy by medical specialty.
natlas_health_out/benchmark_500_ids.csv	The IDs of the 500 held-out questions.
submission/build_pdfs.py	Builds the submission PDFs from the result files.

4. Training setup

Setting	Value
Base model	NCAIR1/N-ATLaS (Llama architecture, 8 billion parameters), loaded in 4-bit NF4
Method	QLoRA: the base weights stay frozen, only low-rank adapter weights are trained
LoRA	rank 16, alpha 32, dropout 0.05, on all attention and MLP projection layers
Trainable parameters	41,943,040 trainable parameters, which is 0.52% of the 8.07 billion parameters in the model
Training examples	3222 single-answer multiple-choice questions
Epochs / learning rate	1 / 0.0002 with a short linear warmup, then cosine decay
Batch	2 per GPU on 2 GPUs with 8 gradient accumulation steps, so an effective batch of 32. 101 optimizer steps, the first 10 as warmup
Loss	Only on the answer (the option letter, plus the expert rationale where the dataset has one)
Hardware and time	Kaggle free tier, 2 x NVIDIA Tesla T4 (16 GB each). Training took 44 minutes. The whole run, including both benchmark passes, took about 51 minutes
Seed	42

5. How to reproduce the run

On Kaggle (free GPU, this is how the reported run was produced):

```
1. Accept the terms for NCAIR1/N-ATLaS and intronhealth/afrimedqa_v2 on Hugging Face.
2. On Kaggle, import natlas_health_kaggle.ipynb as a new notebook.
3. Session options: Accelerator = GPU T4 x2, Internet = On.
4. Add-ons > Secrets: add HF_TOKEN (a Hugging Face write token) and attach it to the notebook.
5. In the first cell, set HF_USERNAME to your own Hugging Face username.
6. Save Version > Save & Run All. Results appear in /kaggle/working/natlas_health_out.
```

On any Linux machine with a 16 GB or larger NVIDIA GPU:

```
pip install "transformers>=4.44" "peft>=0.12" "bitsandbytes>=0.43" "accelerate>=0.33" datasets scipy
export HF_TOKEN=...            # write token with access to both gated repos
export HF_USERNAME=...         # where the adapter is published
export BATCH=2 GRAD_ACCUM=8    # 16 GB GPU. Use BATCH=4 GRAD_ACCUM=4 on 24 GB.
python natlas_pipeline.py      # set PUSH_TO_HUB=0 to skip publishing
```

Library versions used for the reported run: Kaggle GPU notebook image of 12 October 2026 with Python 3.13 and Kaggle's preinstalled PyTorch and transformers (a transformers release that no longer accepts the warmup_ratio argument), plus bitsandbytes installed by the first notebook cell. Exact version numbers were not printed in the run log.

6. How to load and query the model

```
import torch
from transformers import AutoModelForCausalLM, AutoTokenizer, BitsAndBytesConfig
from peft import PeftModel

BASE = "NCAIR1/N-ATLaS"                      # gated: accept the terms on Hugging Face first
ADAPTER = "Emar7/N-ATLaS-Health-QA-LoRA-Powered-by-Awarri"
```

```
tok = AutoTokenizer.from_pretrained(BASE)
bnb = BitsAndBytesConfig(load_in_4bit=True, bnb_4bit_quant_type="nf4",
                         bnb_4bit_use_double_quant=True, bnb_4bit_compute_dtype=torch.float16)
base = AutoModelForCausalLM.from_pretrained(BASE, quantization_config=bnb, device_map={"": 0},
                                            torch_dtype=torch.float16)
model = PeftModel.from_pretrained(base, ADAPTER)   # the adapter sits on top of N-ATLaS
model.eval()

system = ("You are a careful medical assistant answering multiple-choice medical exam questions. "
          "Reply with the letter of the single best option.")
user = ("Question:\nWhich investigation should be done first for a febrile child with pallor and a "
        "palpable spleen in a malaria-endemic area?\n\nOptions:\nA. Chest X-ray\n"
        "B. Blood film for malaria parasites\nC. Lumbar puncture\nD. Urine culture\n\n"
        "Answer with the letter of the single best option.")
messages = [{"role": "system", "content": system}, {"role": "user", "content": user}]
prompt = tok.apply_chat_template(messages, add_generation_prompt=True, tokenize=False,
                                 date_string="11 Oct 2026")
enc = tok(prompt, return_tensors="pt", add_special_tokens=False).to(model.device)
out = model.generate(**enc, max_new_tokens=8, do_sample=False)
print(tok.decode(out[0, enc["input_ids"].shape[1]:], skip_special_tokens=True))
```

This loading path was checked in a fresh Kaggle session (natlas_health_demo.ipynb): the published adapter was downloaded from Hugging Face, loaded on top of N-ATLaS, and its answers to the first five benchmark questions matched the published predictions file 5 out of 5, as did the base model's.

For benchmark-style scoring, compare the next-token logits of the option letters instead of generating text. The function *predict* in natlas_pipeline.py does this.

7. Design decisions

- QLoRA was chosen so the whole run fits on one free 16 GB GPU, which keeps the work reproducible by anyone in Nigeria without paid compute.
- The benchmark is scored by option-letter probability, not by parsing generated text, so no answer is lost to formatting and the score is deterministic.
- The dataset's official train split holds too few usable questions to train on, so 500 questions were held out at random with a fixed seed before any model was scored. This is stated on the model card.
- The adapter is saved straight after training, before re-scoring, so a later failure cannot lose it.

8. Limits and responsible use

- This is a research model. It is not a medical device and must not be used for diagnosis or treatment.
- Training and testing were in English only. Nothing was evaluated in Yoruba, Hausa or Igbo, so no claim is made about those languages.
- Only multiple-choice questions with a single correct answer were used. Questions with more than one correct answer were excluded. Open-ended and consumer questions were not tested.
- The model is scored by which option letter it rates most likely. This measures exam-style knowledge, not the quality or safety of free-text medical advice.
- The training data includes questions from the official AfriMed-QA test split, so this adapter must not be compared with published AfriMed-QA results. The only fair comparison is on the 500 held-out questions.
- One training run, one seed (42), 1 epoch. No hyperparameter search was done, and nothing was tuned on the 500 benchmark questions.
- Both models were scored with 4-bit quantised weights. Full-precision scores may differ slightly.
- The 500 questions come from the same dataset as the training questions. Exact duplicates were removed, but near-duplicate or closely related questions were not checked for.

9. Scaling and next steps

- Evaluate in Yoruba, Hausa and Igbo. N-ATLaS supports these languages, but this adapter has not been tested in them. This needs translated or native medical questions reviewed by clinicians.
- Extend the benchmark to short-answer and consumer health questions with clinician grading, which is closer to real use than multiple choice.
- The adapter is small, so it can be served next to base N-ATLaS on a single GPU and swapped per sector.

10. Licence and attribution

Adapter: N-ATLaS Terms of Use, the same as the base model (research and education, capped at 1,000 active users). N-ATLaS is an initiative of the Federal Ministry of Communications, Innovation and Digital Economy, and powered by Awarri Technologies.

Data: AfriMed-QA v2, CC BY-SA 4.0. Olatunji et al., 2024, arXiv:2411.15640.